

Licenciatura Engenharia Informática e Multimédia
Instituto Superior de Engenharia de Lisboa
Ano letivo 2024/2025

Interação em Ambientes Virtuais

Trabalho Prático Nº3

Turma: 61D

Nome: João Ramos Número: 50730

Nome: Fábio Pestana Número: 50756

Professor: Diogo Cabral

Data: 1 de junho de 2025

Índice

Lista de Figuras 3
Introdução 4
O que é Interação Pessoa-Máquina? 5
Áudio no Unity 5
Audio Scene 6
Microphone Scene 7
Open Sound Control (OSC) 8
Scale Cube TouchOSC 8
Large Language Models 9
Prós e Contras dos Large Language Models 11
LLM Scene 12
Conclusão 14
Bibliografia 15

Lista de Figuras

Figura 1 - Esquema sobre o audio no unity 6
Figura 2 - Simulação da Audio Scene 6
Figura 3 - Simulação da Microphone Scene 7
Figura 4 - Interface na aplicação TouchOSC 8
Figura 5 - Simulação da *scene* Scale Cube TouchOSC 9
Figura 6 - Modelo tradicional de interação entre o jogador e os NPCs 10
Figura 7 - Modelo da interação entre o jogador e os NPCs 10
Figura 8 - Processo de decisão baseado em LLMs 11
Figura 9 - Simulação da situação normal da LLM Scene 12
Figura 10 - Simulação do NPC ajudar o utilizador 12
Figura 11 - Simulação do NPC matar o utilizador 13

Introdução

Este trabalho tem como objetivo a exploração do tema de interação pessoa-máquina com apoio do motor de jogo Unity como plataforma de desenvolvimento. Os alunos tiveram como base o código desenvolvido nas aulas, com o intuito de criar um mundo virtual interativo, ao recorrer a sensores como, por exemplo, o microfone. O foco principal deste projeto é a interação pessoa-máquina que permite capturar e interpretar dados, como sons, de forma a influenciar dinamicamente o comportamento dos objetos no ambiente virtual.

O que é Interação Pessoa-Máquina?

Interação Pessoa-Máquina é uma área da computação que estuda e desenvolve formas de comunicação entre os seres humanos e os sistemas computacionais. O principal objetivo é criar interfaces intuitivas e eficientes de forma a permitir que os utilizadores possam interagir com a tecnologia de forma natural e produtiva.

Tradicionalmente, a interação com sistemas computacionais era realizada através de dispositivos como o teclado, o rato ou interfaces gráficas simples. No entanto, com os avanços tecnológicos e a crescente presença da computação no quotidiano, novas formas de interação tornaram-se possíveis, como o reconhecimento de voz, gestos, movimentos corporais, expressões faciais e até mesmo comandos por linguagem natural.

Estes novos tipos de interação são particularmente relevantes para este trabalho, onde são utilizados sensores como *webcams* ou microfones que desempenham o papel da captação de dados, que depois são processados para modificar dinamicamente o ambiente digital ou o comportamento de objetos virtuais.

A Interação Pessoa-Máquina não foca-se apenas na tecnologia utilizada, mas também na experiência do utilizador. Isto envolve princípios de design centrado no utilizador, ergonomia, acessibilidade e usabilidade. Um bom sistema de Interação Pessoa-Máquina deve ser capaz de interpretar corretamente as intenções do utilizador, fornecer *feedback* adequado e adaptar-se às suas necessidades e limitações.

Áudio no Unity

Este projeto recorre ao sistema de áudio do Unity, que é uma ferramenta integrada no motor de jogo responsável pela reprodução e controlo de sons. O Unity oferece um conjunto de componentes e funcionalidades que permitem criar ambientes sonoros imersivos e responsivos.

O sistema de áudio do Unity é composto por vários elementos principais: os **Audio Sources**, os **Audio Listeners** e os **Audio Clips**. O *Audio Source* é o componente responsável por emitir o som; o *Audio Listener* funciona como o "ouvido" da aplicação, geralmente associado à câmara principal; e o *Audio Clip* representa o ficheiro de áudio propriamente dito, que pode ser um som ambiente, um efeito sonoro ou uma faixa de música.

A comunicação entre os diferentes elementos de áudio é contínua e sincronizada com o ciclo de atualização do Unity. Durante cada *frame*, o sistema de áudio verifica o estado dos objetos sonoros, ajusta os volumes com base na distância entre fontes e o *listener*, e aplica os efeitos definidos. Este ciclo permite que o som reaja dinamicamente ao ambiente virtual, contribuindo para uma experiência mais realista e envolvente.



Figura 1 - Esquema sobre o audio no unity

Audio Scene

A *scene* designada Audio Scene no projeto é um exemplo de como o sistema do Unity pode ser utilizado para criar visualizações de dados sonoros. Esta cena apresenta conceitos de processamento de sinais, onde transforma propriedades do som de uma música em elementos visuais.

A cena oferece três tipos principais de visualização:

- Visualização do espectro: 1024 cubos dispostos em círculo representam diferentes frequências do espectro sonoro.
- Visualização por bandas de frequência: 7 cubos mostram a energia em bandas de frequência específicas, para assim obter-se uma análise mais segmentada do sinal.
- Indicador da energia média do sinal: um cubo que escala verticalmente de acordo com a energia média do sinal do áudio (em decibéis).

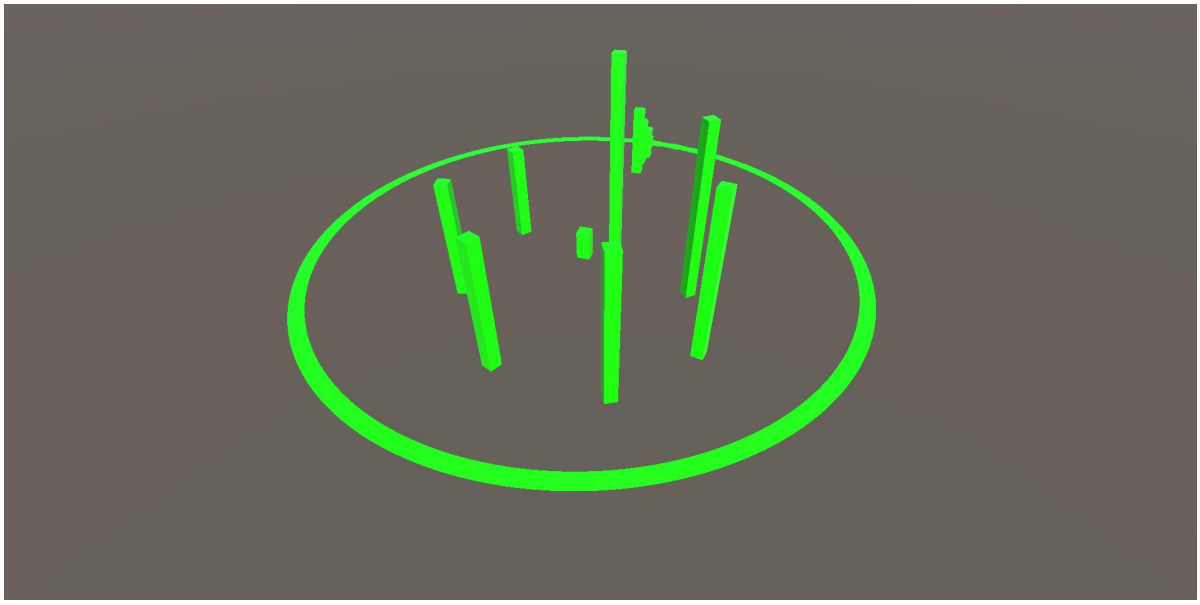


Figura 2 - Simulação da Audio Scene

Na figura anterior, encontra-se a simulação da *scene*, no centro encontra-se o cubo que indica a energia média do sinal (em decibéis), no circulo interior encontram-se 7 cubos, cada um representa uma banda de frequência diferente. Por fim, no circulo exterior, é possível visualizar o espectro do sinal.

Microphone Scene

A *scene* designada por Microphone Scene aplica técnicas de análise do som, com foco na distinção entre assobios e outros sons (como palmas ou vozes). Esta implementação utiliza o microfone do dispositivo como entrada ou um áudio pré-gravado, combinando processamento do sinal com *feedback* visual, quando é identificado um assobio, o agente (uma cápsula) faz uma rotação horizontal de 15 graus, caso seja identificado outro som, o agente move-se para a frente, se nenhum som for identificado o agente fica parado.

Para resolver este problema, desenvolveu-se um sistema que combina análise espectral e deteção de sons para assim classificá-los. Seguiu-se os seguintes passos:

- Deteção de sons: inicialmente calcula-se a energia média do sinal, depois de esta ser convertida para decibéis, utilizou-se o limiar de 80 dB para filtrar os sons ambientais irrelevantes (este valor depende do microfone utilizado). Os sons com valores superiores a 80 dB são considerados os valores relevantes para os agentes.
- Identificação de assobios: depois de um som ser detetado, calcula-se o pico do espectro, ou seja, a frequência dominante do sinal. Em seguida, analisa-se a concentração de energia ao redor desse pico. Se a concentração de energia for superior a 70%, o som é classificado como um assobio, o que leva o agente a rodar 15 graus. Se a concentração for inferior ou igual ao valor anterior, o som é classificado como "outro som", o que leva o agente a deslocar-se para a frente.
- Espera-se 0.5 segundos: realiza-se um *delay* entre as ações do agente para assim evitar que o mesmo som seja identificado várias vezes.

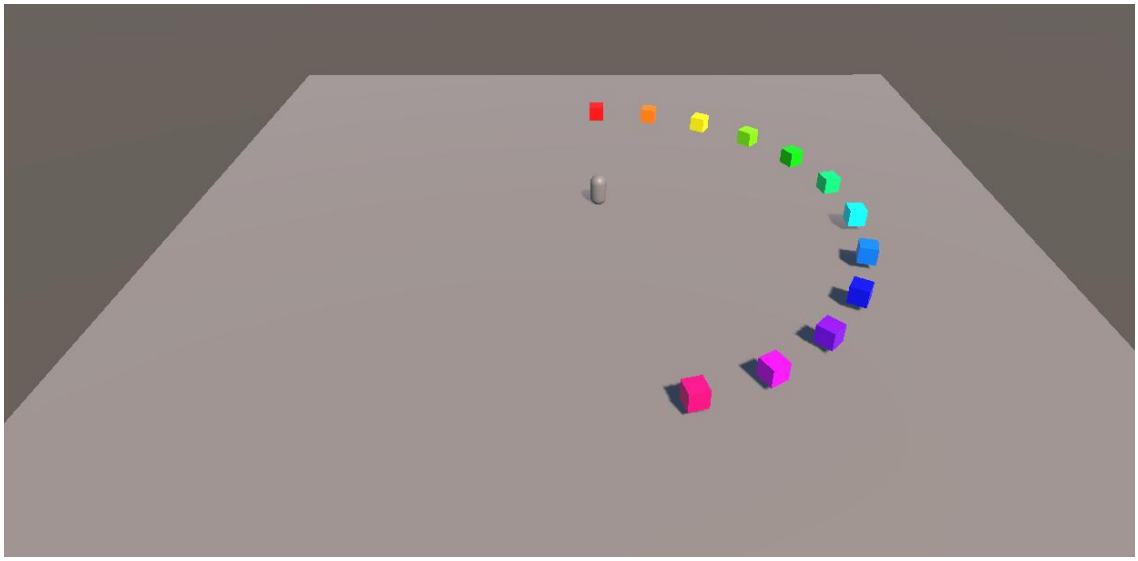


Figura 3 - Simulação da Microphone Scene

Na figura anterior, encontra-se a simulação da *scene*, no centro encontra-se uma cápsula (agente) que espera que o utilizador imita um som para assim classificá-lo.

Open Sound Control (OSC)

O *Open Sound Control* (OSC) é um protocolo de comunicação entre computadores, sintetizadores de som e outros dispositivos multimédia, desenvolvido como uma alternativa mais flexível e moderna ao *Musical Instrument Digital Interface* (MIDI). Diferencia-se pela capacidade de transmitir mensagens estruturadas (com números, vetores, strings, etc) através de redes IP, tornando-o ideal para aplicações interativas e em tempo real.

Scale Cube TouchOSC

Esta *scene* do Unity demonstra uma integração entre dispositivos móveis e o ambiente 3D utilizando o protocolo *Open Sound Control* (OSC). Através da aplicação **TouchOSC** (instalada num telemóvel), os utilizadores podem controlar e interagir com um cubo virtual em tempo real, explorando dois modos distintos de manipulação:

- ***Slider***: controla a escala vertical do cubo através de um *slider* nas mensagens que o servidor (Unity) recebe do telemóvel atualiza-se a escala do cubo segundo o valor do `float` da mensagem "`/1/fader1`" .
- **Acelerómetro**: utiliza os dados do acelerómetro do telemóvel ("`/accxyz`") para alterar a escala do cubo conforme a posição física do dispositivo, ou seja, telemóvel deitado, o valor **y** da escala do cubo fica com o valor 0.1, telemóvel "em pé", o valor **z** da escala do cubo fica com o valor 0.1 e telemóvel de lado, o valor **x** da escala do cubo fica com o valor 0.1.

Para alternar-se entre os dois modos, na interface do telemóvel encontra-se um botão, caso o valor que este retorne seja 1 (botão "pintado" ou ativado), utiliza-se o *slider*, se o valor for 0 (botão desativado) utiliza-se o valor do acelerómetro. Na seguinte imagem encontra-se a interface utilizada no telemóvel (*slider* amarelo do lado esquerdo altera a escala do cubo e o botão embaixo permite a alternância entre os modos).

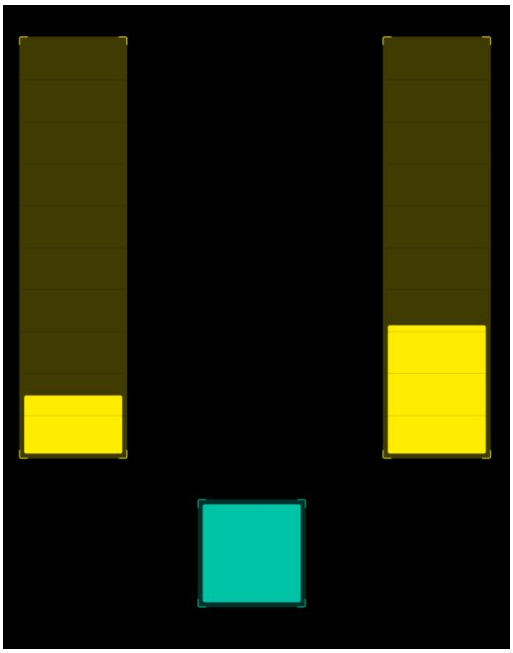


Figura 4 - Interface na aplicação TouchOSC

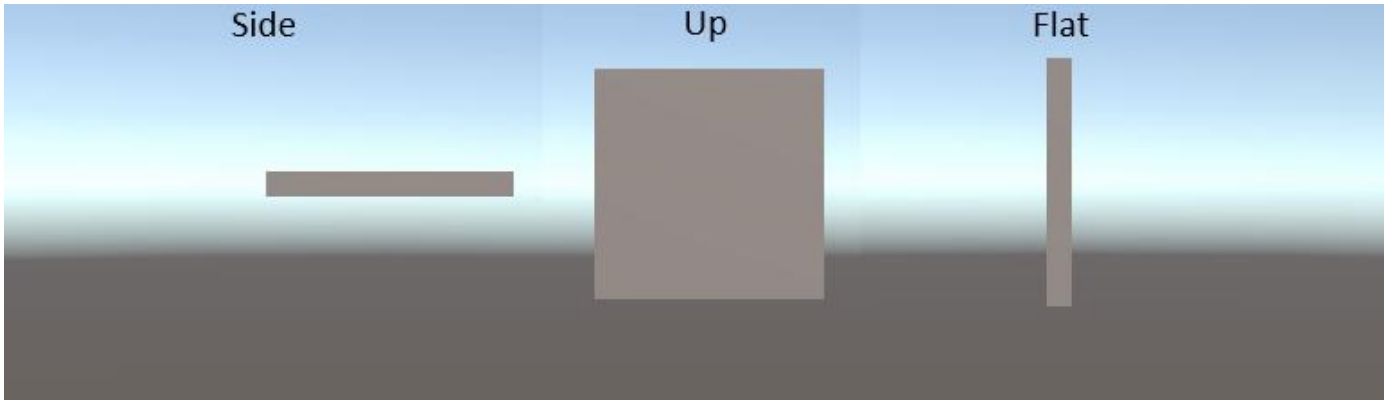


Figura 5 - Simulação da *scene* Scale Cube TouchOSC

Na figura anterior, encontra-se a simulação da *scene*, demonstrando os vários modos do cubo, quando este encontra-se no modo do acelerómetro (por cima de cada simulação encontra-se a orientação identificada).

Large Language Models

Os LLMs (*Large Language Models*) são redes neurais treinadas com enormes quantidades de texto para compreender, gerar e manipular linguagem natural de forma orgânica. Estes modelos são capazes de interpretar o significado de frases, manter contextos de conversação, responder a perguntas, traduzir textos, gerar conteúdos coerentes e até simular diálogos realistas entre humanos e máquinas.

Estes modelos funcionam com base em arquiteturas de *transformers* que utilizam mecanismos de atenção para analisar a relação entre palavras e prever a continuação mais provável de uma sequência de texto. Devido ao seu grande número de parâmetros e ao volume massivo de dados usados durante o treino, as LLMs conseguem captar nuances da linguagem e produzir respostas contextualmente adequadas.

No contexto deste projeto, as LLMs são utilizadas para implementar interação por linguagem natural entre o jogador e personagens não-jogáveis (NPCs) dentro do mundo virtual criado no Unity. A integração é realizada através da plataforma Ollama, que permite executar modelos de linguagem localmente, o que facilita a comunicação entre o motor de jogo e o modelo de linguagem.

Através desta abordagem, o jogador pode interagir com os NPCs ao dar comandos em linguagem natural. As mensagens são processadas pela LLM, que gera uma resposta adequada, o que pode influenciar diretamente o comportamento da personagem ou alterar elementos do ambiente.

Para ilustrar de forma mais clara a diferença entre abordagens tradicionais e abordagens baseadas em LLMs na definição do comportamento dos agentes, apresentam-se a seguir três diagramas explicativos:

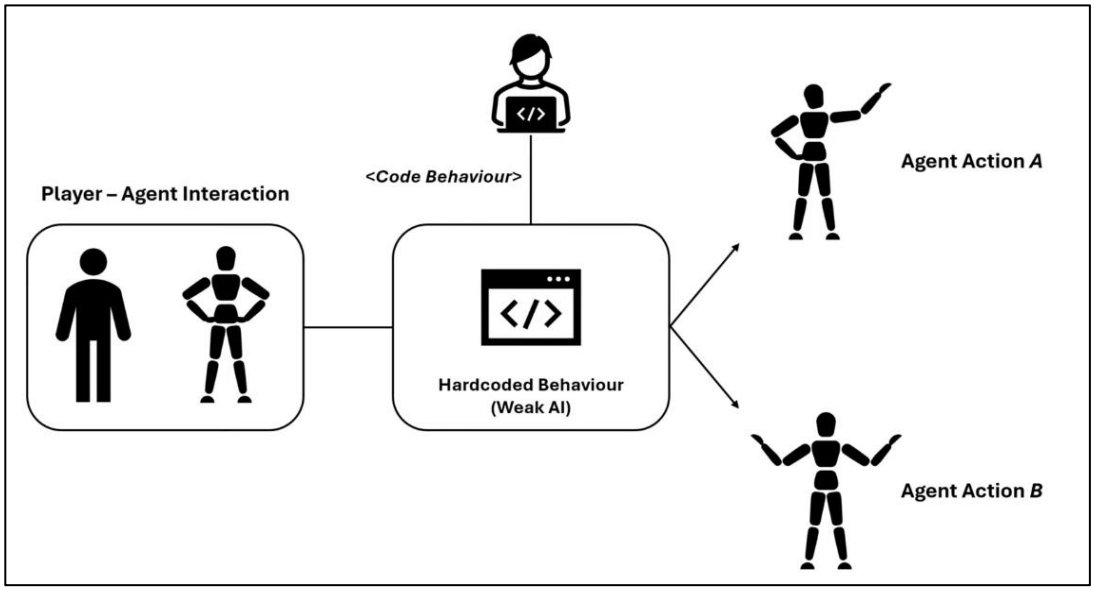


Figura 6 - Modelo tradicional de interação entre o jogador e os NPCs

Esta primeira imagem representa um modelo tradicional de interação entre o jogador e os NPCs, onde o comportamento dos agentes é definido através de código *hardcoded*. Neste exemplo, o programador antecipa todas as possíveis ações e respostas do agente, o que resulta numa IA fraca, com comportamento limitado e previsível.

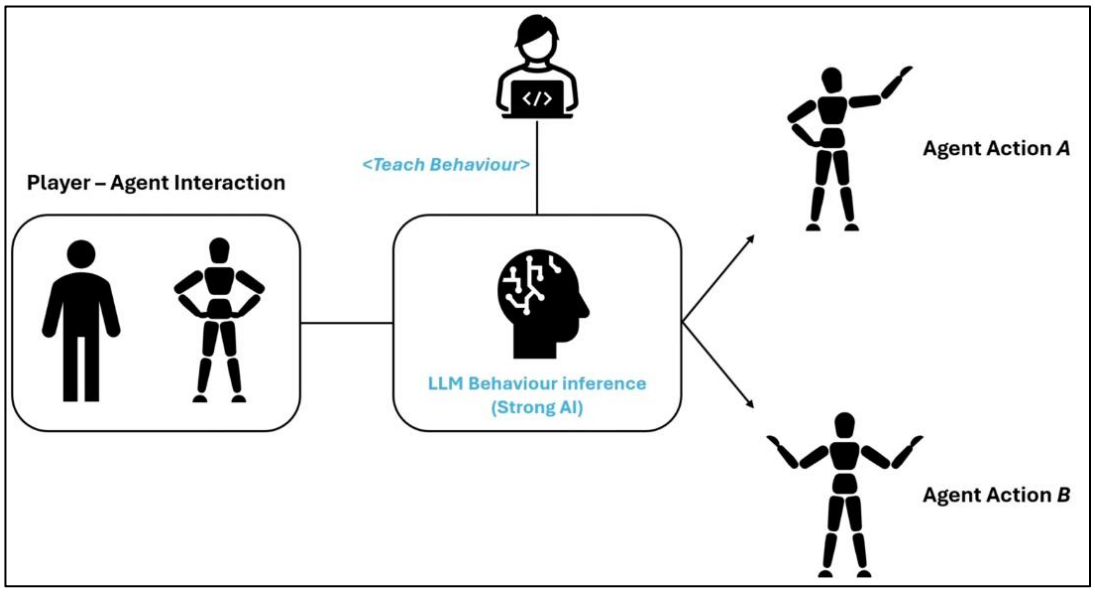


Figura 7 - Modelo da interação entre o jogador e os NPCs

Esta imagem mostra um modelo mais avançado de interação, no qual a lógica dos agentes é feita por um LLM (*Large Language Model*). Em vez de respostas programadas, os agentes aprendem a comportar-se de acordo com exemplos e instruções fornecidas. Este modelo permite respostas mais flexíveis e contextualmente relevantes, o que resulta numa IA mais forte.

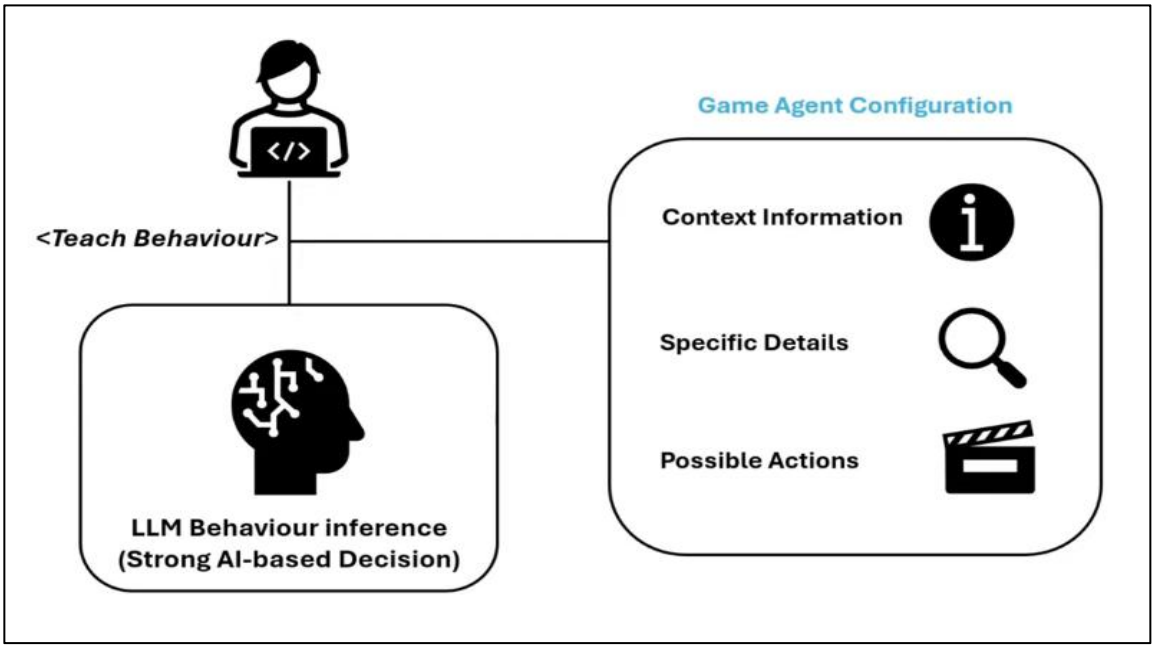


Figura 8 - Processo de decisão baseado em LLMs

Por fim, esta imagem aprofunda o processo de decisão baseado em LLMs, nomeadamente como os programadores ensinam o comportamento dos agentes ao fornecer informações de contexto, detalhes específicos e ações possíveis. Com base nestes dados, o modelo de linguagem toma decisões informadas, de forma a tomar comportamentos realistas e dinâmicos no ambiente virtual.

Prós e Contras dos Large Language Models

A utilização de LLMs para definir o comportamento de NPCs em jogos traz várias vantagens significativas. Em primeiro lugar, permite uma jogabilidade mais dinâmica, uma vez que os agentes conseguem responder de forma contextualizada às interações do jogador. Esta abordagem reduz substancialmente a quantidade de código necessária para definir os comportamentos, o que facilita o desenvolvimento e manutenção do projeto. Além disso, alterações no comportamento dos agentes podem ser feitas com menor impacto direto no código. Por fim, a definição de múltiplos agentes com comportamentos distintos torna-se mais simples, uma vez que os modelos podem adaptar-se a diferentes perfis e contextos apenas com mudanças no *prompt* ou nos dados fornecidos.

No entanto, esta abordagem também apresenta algumas desvantagens. Um dos principais problemas é a possibilidade de o modelo "alucinar", ou seja, gerar respostas que parecem coerentes mas são incorretas ou irrelevantes para o contexto. Além disso, os LLMs podem produzir comportamentos inesperados, especialmente em situações mal definidas ou ambíguas, o que pode comprometer a consistência da experiência de jogo. Por fim, os testes tornam-se menos determinísticos, já que a mesma entrada pode gerar respostas ligeiramente diferentes em execuções distintas, dificultando a validação e depuração do sistema.

LLM Scene

A LLM Scene utiliza *Large Language Models* (LLMs) para trazer interações entre o utilizador e os agentes com o LLM. Esta cena, demonstra como integrar modelos de linguagem (Ollama) diretamente no Unity, permitindo que NPCs compreendam e respondam a comandos em linguagem natural de forma contextualizada, dependendo das interações entre o utilizador e os agentes, estes podem ajudá-lo, abrindo-lhe uma porta (agentes conseguem alterar o ambiente, neste caso conseguem abrir uma porta) ou recusar-se a abri-la e até mesmo matar o utilizador.

O ponto principal desta *scene* é a ligação local à plataforma Ollama, que executa modelos de linguagem (utilizou-se o modelo da Meta Llama 3.2) no dispositivo do utilizador, sem a necessidade de APIs externas. Através disto os NPCs na *scene* conseguem processar os inputs de texto do utilizador e conseguem respondê-lo.

Nas seguintes imagens encontram-se respetivamente os exemplos: situação normal, NPC ajudar o utilizador e abrir a porta e o NPC matar o utilizador.

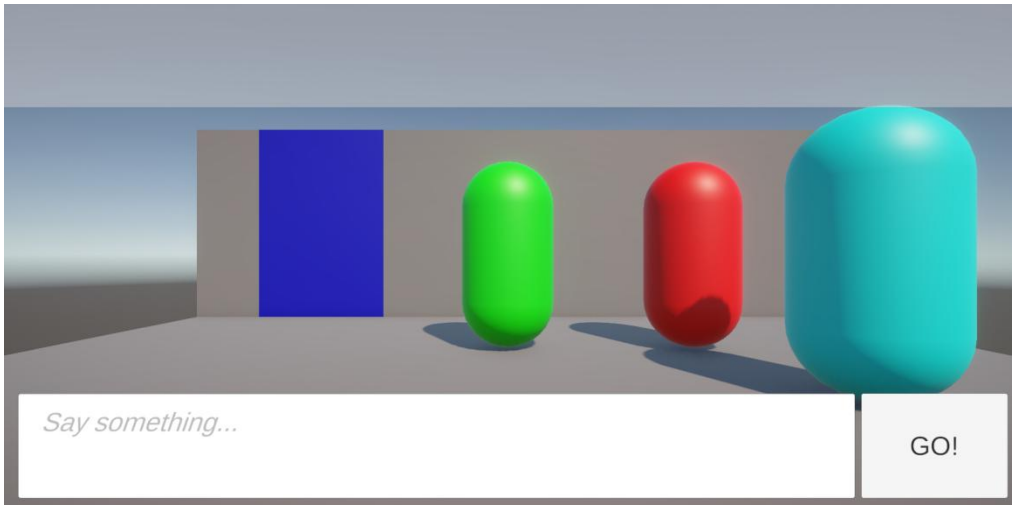


Figura 9 - Simulação da situação normal da LLM Scene

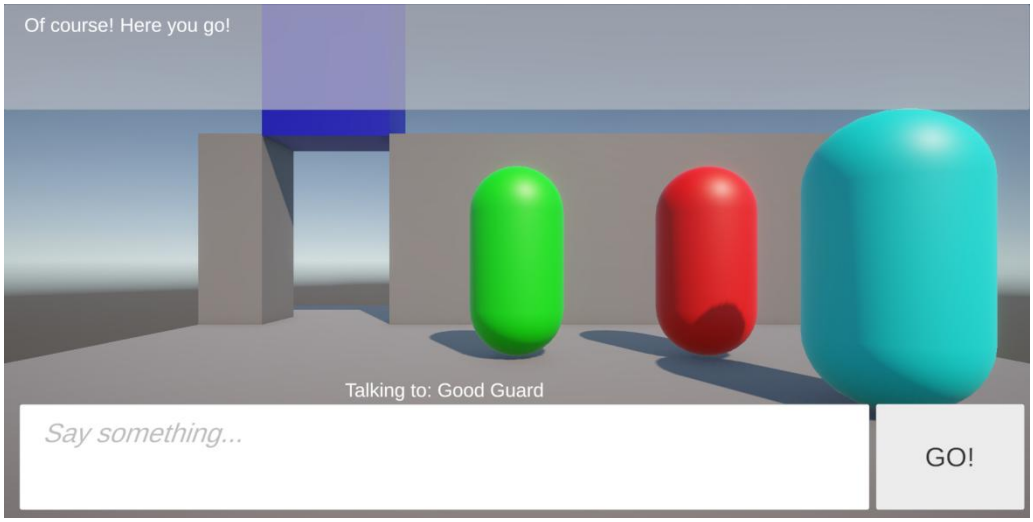


Figura 10 - Simulação do NPC ajudar o utilizador



Figura 11 - Simulação do NPC matar o utilizador

Conclusão

O desenvolvimento deste projeto permitiu-nos explorar de forma prática o tema da interação pessoa-máquina em ambientes virtuais, utilizando o motor de jogo Unity. A integração de sensores e, principalmente, a utilização de LLMs (*Large Language Models*) para a definição do comportamento de NPCs são abordagens inovadora e interessantes para criar experiências mais naturais e imersivas.

Para trabalhos futuros, seria interessante aprofundar estas interações em jogos mais complexos.

Bibliografia

Consulta:

Vídeos gravados pelo Prof. Arnaldo Abrantes - Moodle

Wikipedia. (2023, April 9). Large language model. Wikipedia. https://en.wikipedia.org/wiki/Large_language_model

Jay's Hub. (2025, March 9). Let a Large Language Model (LLM) take Decisions in a Unity Game at No Cost. YouTube. https://www.youtube.com/watch?v=qhqPPNiNst0